

Algoritmo FFDLS para el Problema de Empaquetado en Bins de Tamaño Variable

Resumen

El algoritmo **FFDLS** (First Fit Decreasing using Largest bins with Shifting) resuelve una variante del *Variable Sized Bin Packing Problem (VSBPP)*, donde se busca empaquetar ítems de distintos tamaños en bins de varias capacidades disponibles. El objetivo es minimizar el **costo total**, entendido como la suma de las capacidades de los bins utilizados.

Este algoritmo emplea una heurística en dos fases:

1. Una primera fase de asignación usando **First Fit Decreasing** (FFD) con bins del tamaño más grande disponible.
2. Una segunda fase de **shifting** o ajuste de bins, donde se intentan reducir los bins asignados a uno más pequeño si es posible.

Entrada

- **items**: Lista de tamaños de los ítems a empaquetar.
- **bin_sizes**: Lista de capacidades posibles de bins (sin límite en la cantidad de cada tipo).

Salida

Una lista de bins representados como tuplas (**capacidad**, **utilizado**, **[contenidos]**).

Función Objetivo

Minimizar el costo total:

$$\text{Costo Total} = \sum_{b \in B} \text{capacidad}_b$$

Heurística de Reducción (Shifting)

Luego de colocar un ítem en un bin, si:

- Algún ítem en ese bin es mayor a $\frac{1}{3}$ del bin más grande (`max_capacity`),
- El bin está infrautilizado (se usa $\leq$ que un bin más pequeño disponible),

entonces se intenta reemplazar dicho bin por el más pequeño posible que aún pueda contener los ítems ya asignados.

Pseudocódigo

Listing 1: Algoritmo FFDLS

```
1 1. Ordenar items de mayor a menor.
2 2. Ordenar bin_sizes de mayor a menor.
3 3. Inicializar bins vac\'\{i}os.
4
5 4. Para cada item en items:
6     a. Intentar colocarlo en el primer bin donde quepa.
7     b. Si se coloca:
8         i. Aplicar shifting:
9             - Buscar si puede ser contenido en un bin m\'as peque\~no.
10    c. Si no se coloca:
11        i. Crear un nuevo bin del tipo m\'as grande disponible.
12
13 5. Reempaquetar todos los \'\{i}tems resultantes usando Best Fit
    Decreasing:
14    a. Intentar colocar cada item en el bin que minimice el espacio
        restante.
15    b. Si no cabe en ning\'un bin existente:
16        - Abrir uno nuevo con la menor capacidad posible que lo
            contenga.
```

Implementación en Python

La implementación completa está disponible en el siguiente bloque:

```
1 def ffdls(items, bin_sizes):
2     def try_shift(bin_idx, bins, bin_sizes, max_capacity):
3         cap, space, contents = bins[bin_idx]
4         if any(i > (1/3) * max_capacity for i in contents):
5             bin_used = cap - space
6             candidate_bins = [c for c in bin_sizes if c < cap and c >=
7                               (3/4) * cap]
8             if candidate_bins:
9                 smallest_fit = min(candidate_bins)
10                if bin_used <= smallest_fit:
11                    bins[bin_idx] = (smallest_fit, smallest_fit -
12                                      bin_used, contents.copy())
13
14    items = sorted(items, reverse=True)
15    bin_sizes = sorted(bin_sizes, reverse=True)
16    max_capacity = bin_sizes[0]
```

```

15     bins = []
16
17     for item in items:
18         placed = False
19         for idx, (cap, space, contents) in enumerate(bins):
20             if space >= item:
21                 contents = contents.copy()
22                 contents.append(item)
23                 bins[idx] = (cap, space - item, contents)
24                 try_shift(idx, bins, bin_sizes, max_capacity)
25                 placed = True
26                 break
27         if not placed:
28             bins.append((max_capacity, max_capacity - item, [item]))
29
30     all_items = []
31     for _, _, contents in bins:
32         all_items.extend(contents)
33     all_items = sorted(all_items, reverse=True)
34
35     final_bins = []
36     bin_sizes_sorted = sorted(bin_sizes)
37
38     for item in all_items:
39         best_fit = -1
40         min_space_left = float('inf')
41         for idx, (cap, space, contents) in enumerate(final_bins):
42             if space >= item and (space - item) < min_space_left:
43                 best_fit = idx
44                 min_space_left = space - item
45         if best_fit != -1:
46             cap, space, contents = final_bins[best_fit]
47             contents = contents.copy()
48             contents.append(item)
49             final_bins[best_fit] = (cap, space - item, contents)
50         else:
51             for cap in bin_sizes_sorted:
52                 if cap >= item:
53                     final_bins.append((cap, cap - item, [item]))
54                     break
55             else:
56                 raise Exception(f"Item {item} no cabe en ning\un bin disponible")
57
58     return [(cap, cap - space, contents) for cap, space, contents in
            final_bins]

```

Métrica de Evaluación

La función `print_bin_summary` muestra la ocupación de cada bin y el costo total (suma de capacidades). Esta métrica es útil cuando las capacidades están asociadas a un costo proporcional.